Lab 7: Fuzzing for Input Validation Bugs

Name: Subhransi
SRN: PES1UG24CS161
Assigned module: L6_L7_33_spam_scoring (Spam Scoring Filter)

Objective

To explore fuzz testing as a security and quality assurance technique by identifying and fixing bugs in the provided Python script (`spam_scoring.py`) using the Hypothesis fuzzing library.

About the module under test

`spam_scoring.py` has three small functions used when scoring a message as spam:

Function	Purpose	Expected input
sender_domain(email)	Returns the lowercase domain of a sender address	e-mail string such as offers@Deals.com
first_link(text)	Returns the first link found in a message	message text containing http:// or https://
shout_ratio(text)	Fraction of letters that are uppercase (measures SHOUTING)	message text such as FREE prize NOW

The original code assumes every input is well formed. Fuzzing is used to find what happens when it is not (empty strings, None, missing characters, odd whitespace and so on).

Step 1: Install dependencies

Commands: `pip install hypothesis` and `pip install pytest`.
Hypothesis is the property-based fuzzing library that generates the random inputs, and pytest is the test runner.

```
PS C:\Users\Epari Subhransi\SE_lab_assignments_PES1UG24CS161\SE_lab_7> pip install hypothesis
Requirement already satisfied: hypothesis in c:\users\epari subhransi\python313\lib\site-packages (6.170.15)
Requirement already satisfied: sortedcontainers<3.0.0,>=2.1.0 in c:\users\epari subhransi\python313\lib\site-packages (from hypothesis) (2.4.0)

[notice] A new release of pip is available: 25.3 -> 26.3
[notice] To update, run: python.exe -m pip install --upgrade pip
PS C:\Users\Epari Subhransi\SE_lab_assignments_PES1UG24CS161\SE_lab_7> pip install pytest
Requirement already satisfied: pytest in c:\users\epari subhransi\python313\lib\site-packages (9.2.0)
Requirement already satisfied: colorama>=0.4 in c:\users\epari subhransi\appdata\roaming\python\python313\site-packages (from pytest) (0.4.6)
Requirement already satisfied: iniconfig>=2 in c:\users\epari subhransi\python313\lib\site-packages (from pytest) (2.3.1)
Requirement already satisfied: packaging>=24 in c:\users\epari subhransi\appdata\roaming\python\python313\site-packages (from pytest) (25.0)
Requirement already satisfied: pluggy<2,>=1.6 in c:\users\epari subhransi\python313\lib\site-packages (from pytest) (1.7.0)
Requirement already satisfied: pygments>=2.15 in c:\users\epari subhransi\appdata\roaming\python\python313\site-packages (from pytest) (2.19.2)

[notice] A new release of pip is available: 25.3 -> 26.3
[notice] To update, run: python.exe -m pip install --upgrade pip
PS C:\Users\Epari Subhransi\SE_lab_assignments_PES1UG24CS161\SE_lab_7>
```

Figure 1: Installing Hypothesis and pytest

Step 2: Run the buggy application

Command: `python spam_scoring.py` (original, unfixed script). With valid input the program works; with invalid input it crashes.

```
PS C:\Users\Epari Subhransi\SE_lab_assignments_PES1UG24CS161\SE_lab_7> python spam_scoring.py
==== Running sender_domain ====
Enter a sender email (e.g. offers@Deals.com): offers@Deals.com
Sender Domain: deals.com

==== Running first_link ====
Enter a message containing a link: Visit https://google.com today
First Link: https://google.com

==== Running shout_ratio ====
Enter a message (e.g. FREE prize NOW): FREE prize NOW
Uppercase Ratio: 0.5833333333333334
PS C:\Users\Epari Subhransi\SE_lab_assignments_PES1UG24CS161\SE_lab_7>
```

Figure 2: Original script with valid input (works)

The same script was then run with invalid input. At the first prompt the input `nobody` (an e-mail with no @) was entered:

```
PS C:\Users\Epari Subhransi\SE_lab_assignments_PES1UG24CS161\SE_lab_7> python spam_scoring.py
==== Running sender_domain ====
Enter a sender email (e.g. offers@Deals.com): nobody
Traceback (most recent call last):
  File "C:\Users\Epari Subhransi\SE_lab_assignments_PES1UG24CS161\SE_lab_7\spam_scoring.py", line 33, in <module>
    main()
    ~~~~^^
  File "C:\Users\Epari Subhransi\SE_lab_assignments_PES1UG24CS161\SE_lab_7\spam_scoring.py", line 19, in main
    result = sender_domain(user_input)
  File "C:\Users\Epari Subhransi\SE_lab_assignments_PES1UG24CS161\SE_lab_7\spam_scoring.py", line 2, in sender_domain
    return email.split('@')[1].lower()
           ~~~~~~~~~~~~~~~~~~~~^^^
IndexError: list index out of range
PS C:\Users\Epari Subhransi\SE_lab_assignments_PES1UG24CS161\SE_lab_7>
```

Figure 3: Original script crashing with IndexError in sender_domain for input 'nobody'

Because `main()` stops at the first crash, the other two functions were called directly to show their failures:

```
python -c "from spam_scoring import first_link; print(first_link('no link here'))"
python -c "from spam_scoring import shout_ratio; print(shout_ratio('123'))"
```

```
PS C:\Users\Epari Subhransi\SE_lab_assignments_PES1UG24CS161\SE_lab_7> python -c "from spam_scoring import first_link; print(first_link('no link here'))"
Traceback (most recent call last):
  File "<string>", line 1, in <module>
    from spam_scoring import first_link; print(first_link('no link here'))
                                       ~~~~~~~~~~^^^^^^^^^^^^^^^^
  File "C:\Users\Epari Subhransi\SE_lab_assignments_PES1UG24CS161\SE_lab_7\spam_scoring.py", line 5, in first_link
    start = text.index('http')
ValueError: substring not found
```

Figure 4: Original first_link crashing with ValueError when the message has no link

```
PS C:\Users\Epari Subhransi\SE_lab_assignments_PES1UG24CS161\SE_lab_7> python -c "from spam_scoring import shout_ratio; print(shout_ratio('123'))"
Traceback (most recent call last):
  File "<string>", line 1, in <module>
    from spam_scoring import shout_ratio; print(shout_ratio('123'))
                                        ~~~~~~~~~~^^^^^^^
  File "C:\Users\Epari Subhransi\SE_lab_assignments_PES1UG24CS161\SE_lab_7\spam_scoring.py", line 14, in shout_ratio
    return len(upper) / len(letters)
           ~~~~~~~~~~~^~~~~~~~~~~~~
ZeroDivisionError: division by zero
PS C:\Users\Epari Subhransi\SE_lab_assignments_PES1UG24CS161\SE_lab_7>
```

Figure 5: Original shout_ratio crashing with ZeroDivisionError when the message has no letters

Step 3: Complete test_spam_scoring.py using Hypothesis

The three starter tests (`test_sender_domain_no_crash`, `test_first_link_no_crash`, `test_shout_ratio_no_crash`) feed `st.text() | st.none()` to a function and fail if it raises any exception. I completed them and added `@example` inputs for known edge cases (None, empty string, no @, no link, no letters), property tests and sanity checks. The completed file has 14 tests:

Function	Test	What it checks
sender_domain	test_sender_domain_no_crash	Never raises for any text or None
	test_sender_domain_returns_clean_value	For any input (text, None, numbers, lists) the result is None or a non-empty lowercase string without '@' or whitespace
Function	**Test**	**What it checks**
	test_sender_domain_valid_email_gives_lowercase_domain	user@Domain.com always gives 'domain.com'

	test_sender_domain_rejects_missing_parts	No @, missing user, missing domain and two @ signs all give None
first_link	test_first_link_no_crash	Never raises for any text or None
	test_first_link_returns_clean_value	Result is None or a link that appears in the text and has no whitespace
	test_first_link_text_without_a_link_gives_none	Text with no link (including 'nothttp' and 'see http') gives None
	test_first_link_finds_link_and_stops_at_whitespace	The link ends at a space or a newline
shout_ratio	test_shout_ratio_no_crash	Never raises for any text or None
	test_shout_ratio_is_between_0_and_1	Result is always a float from 0.0 to 1.0
	test_shout_ratio_text_without_letters_is_zero	Digits, punctuation and spaces only gives 0.0
	test_shout_ratio_all_caps_is_one	All uppercase letters gives 1.0
	test_shout_ratio_all_lowercase_is_zero	All lowercase letters gives 0.0
all	test_known_examples	Normal examples such as offers@Deals.com still work

Step 4: Run test_spam_scoring.py against the buggy script

Command: `python -m pytest test_spam_scoring.py`. Result on the original script: **10 failed, 4 passed**. Hypothesis prints the smallest failing input for each failure (for example `s=''` and `s=None`).

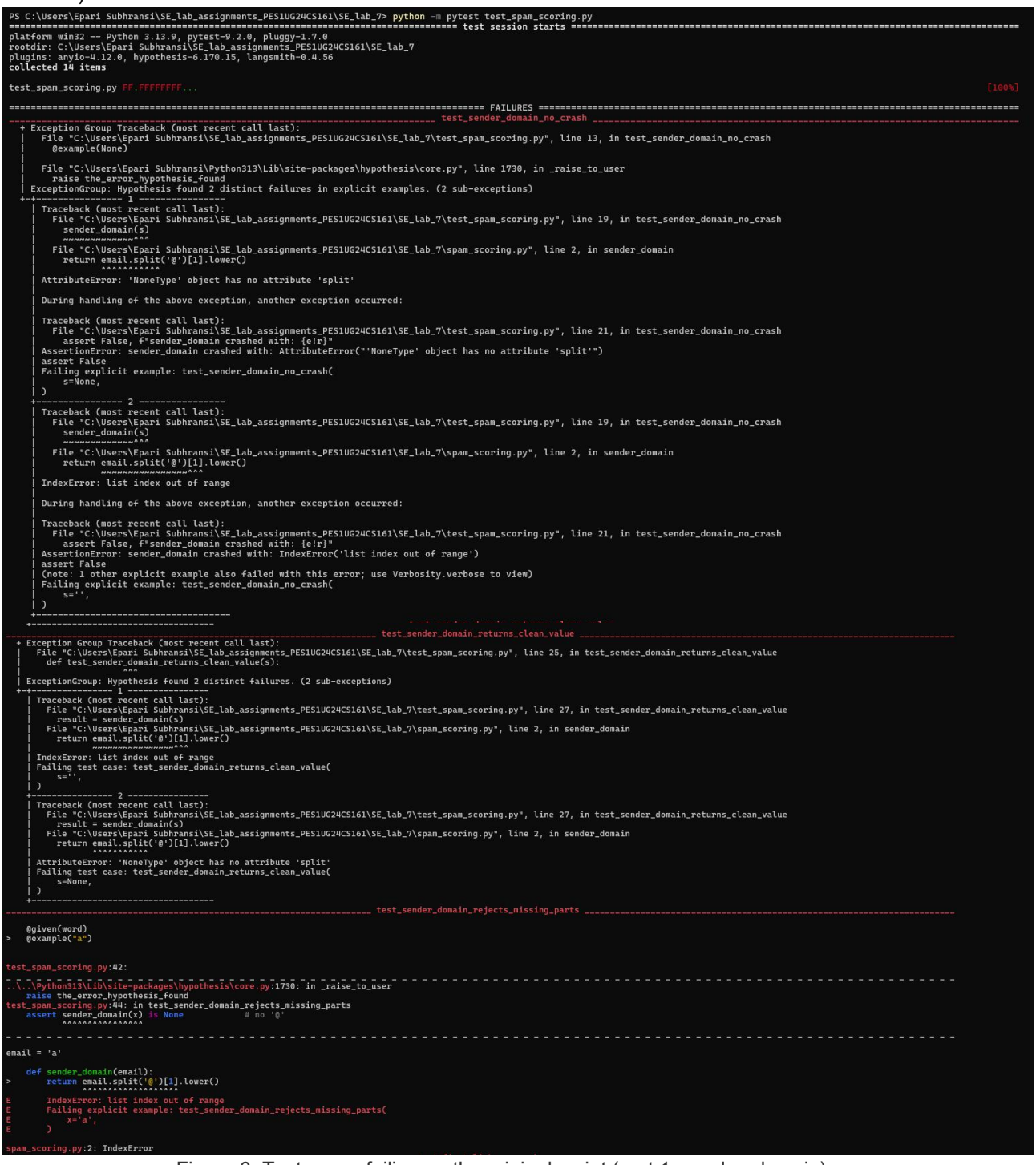


Figure 6: Test cases failing on the original script (part 1, sender_domain)

```
_____ test_first_link_no_crash _____
  + Exception Group Traceback (most recent call last):
  |   File "C:\Users\Epari Subhransi\SE_lab_assignments_PES1UG24CS161\SE_lab_7\test_spam_scoring.py", line 52, in test_first_link_no_crash
  |     @example(None)
  |
  |   File "C:\Users\Epari Subhransi\Python313\Lib\site-packages\hypothesis\core.py", line 1730, in _raise_to_user
  |     raise the_error_hypothesis_found
  | ExceptionGroup: Hypothesis found 2 distinct failures in explicit examples. (2 sub-exceptions)
  +-+---------------- 1 ----------------
    | Traceback (most recent call last):
    |   File "C:\Users\Epari Subhransi\SE_lab_assignments_PES1UG24CS161\SE_lab_7\test_spam_scoring.py", line 57, in test_first_link_no_crash
    |     first_link(s)
    |     ~~~~~~~~~~^^^
    |   File "C:\Users\Epari Subhransi\SE_lab_assignments_PES1UG24CS161\SE_lab_7\spam_scoring.py", line 5, in first_link
    |     start = text.index('http')
    |             ^^^^^^^^^^
    | AttributeError: 'NoneType' object has no attribute 'index'
    |
    | During handling of the above exception, another exception occurred:
    |
    | Traceback (most recent call last):
    |   File "C:\Users\Epari Subhransi\SE_lab_assignments_PES1UG24CS161\SE_lab_7\test_spam_scoring.py", line 59, in test_first_link_no_crash
    |     assert False, f"first_link crashed with: {e!r}"
    | AssertionError: first_link crashed with: AttributeError("'NoneType' object has no attribute 'index'")
    | assert False
    | Failing explicit example: test_first_link_no_crash(
    |     s=None,
    | )
    +---------------- 2 ----------------
    | Traceback (most recent call last):
    |   File "C:\Users\Epari Subhransi\SE_lab_assignments_PES1UG24CS161\SE_lab_7\test_spam_scoring.py", line 57, in test_first_link_no_crash
    |     first_link(s)
    |     ~~~~~~~~~~^^^
    |   File "C:\Users\Epari Subhransi\SE_lab_assignments_PES1UG24CS161\SE_lab_7\spam_scoring.py", line 5, in first_link
    |     start = text.index('http')
    | ValueError: substring not found
    |
    | During handling of the above exception, another exception occurred:
    |
    | Traceback (most recent call last):
    |   File "C:\Users\Epari Subhransi\SE_lab_assignments_PES1UG24CS161\SE_lab_7\test_spam_scoring.py", line 59, in test_first_link_no_crash
    |     assert False, f"first_link crashed with: {e!r}"
    | AssertionError: first_link crashed with: ValueError('substring not found')
    | assert False
    | (note: 1 other explicit example also failed with this error; use Verbosity.verbose to view)
    | Failing explicit example: test_first_link_no_crash(
    |     s='',
    | )
    +------------------------------------
_____ test_first_link_returns_clean_value _____
  + Exception Group Traceback (most recent call last):
  |   File "C:\Users\Epari Subhransi\SE_lab_assignments_PES1UG24CS161\SE_lab_7\test_spam_scoring.py", line 63, in test_first_link_returns_clean_value
  |     def test_first_link_returns_clean_value(s):
  |                ^^^
  | ExceptionGroup: Hypothesis found 2 distinct failures. (2 sub-exceptions)
  +-+---------------- 1 ----------------
    | Traceback (most recent call last):
    |   File "C:\Users\Epari Subhransi\SE_lab_assignments_PES1UG24CS161\SE_lab_7\test_spam_scoring.py", line 65, in test_first_link_returns_clean_value
    |     result = first_link(s)
    |   File "C:\Users\Epari Subhransi\SE_lab_assignments_PES1UG24CS161\SE_lab_7\spam_scoring.py", line 5, in first_link
    |     start = text.index('http')
    | ValueError: substring not found
    | Failing test case: test_first_link_returns_clean_value(
    |     s='',  # or any other generated value
    | )
    +---------------- 2 ----------------
    | Traceback (most recent call last):
    |   File "C:\Users\Epari Subhransi\SE_lab_assignments_PES1UG24CS161\SE_lab_7\test_spam_scoring.py", line 65, in test_first_link_returns_clean_value
    |     result = first_link(s)
    |   File "C:\Users\Epari Subhransi\SE_lab_assignments_PES1UG24CS161\SE_lab_7\spam_scoring.py", line 5, in first_link
    |     start = text.index('http')
    |             ^^^^^^^^^^
    | AttributeError: 'NoneType' object has no attribute 'index'
    | Failing test case: test_first_link_returns_clean_value(
    |     s=None,  # or any other generated value
    | )
    +------------------------------------
_____ test_first_link_text_without_a_link_gives_none _____

    @given(st.text(alphabet=string.ascii_letters + " ").filter(lambda t: "http" not in t.lower()))
>   @example("nothttp is a word")
                ^^^

test_spam_scoring.py:74:
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
..\..\Python313\Lib\site-packages\hypothesis\core.py:1730: in _raise_to_user
    raise the_error_hypothesis_found
- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -

s = 'see http'

    @given(st.text(alphabet=string.ascii_letters + " ").filter(lambda t: "http" not in t.lower()))
    @example("nothttp is a word")
    @example("see http")
    def test_first_link_text_without_a_link_gives_none(s):
>       assert first_link(s) is None
E       AssertionError: assert 'http' is None
E        +  where 'http' = first_link('see http')
E       (note: 1 other explicit example also failed with this error; use Verbosity.verbose to view)
E       Failing explicit example: test_first_link_text_without_a_link_gives_none(
E           s='see http',
E       )

test_spam_scoring.py:77: AssertionError
_____ test_first_link_finds_link_and_stops_at_whitespace _____

    @given(word, word)
>   def test_first_link_finds_link_and_stops_at_whitespace(path, after):
                ^^^

test_spam_scoring.py:81:
- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
```

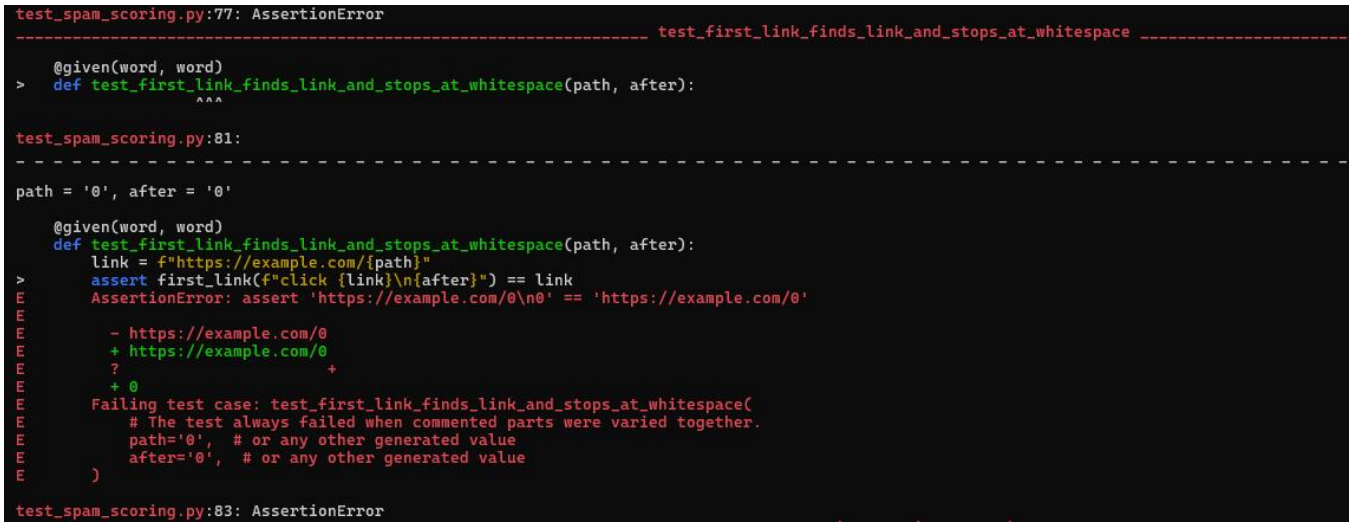


Figure 7: Test cases failing on the original script (part 2, first_link)



Figure 8: Test cases failing on the original script (part 3, shout_ratio)

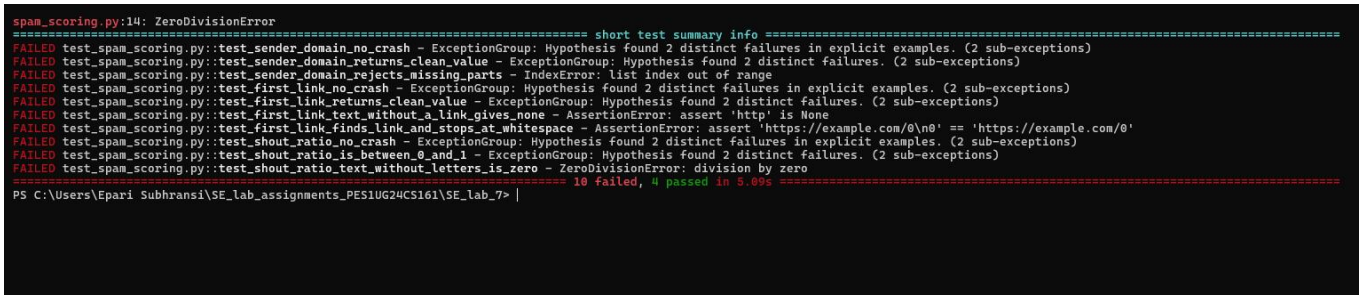


Figure 9: Short test summary on the original script: 10 failed, 4 passed

Bugs found

Fuzzing found both crashes and silent wrong answers, where the function returns a value that looks fine but is incorrect. The silent ones are the more dangerous kind because nothing warns the caller.

Function	Failing input	Original behaviour	Fix
sender_domain	None, 123	AttributeError (no .split on None/int)	Non-string input returns None
sender_domain	'', 'nobody'	IndexError (no @ so split()[1] fails)	Require exactly one @, else None
sender_domain	'user@'	Returns '' (empty domain, no error)	Empty domain returns None
sender_domain	'a@b@c.com'	Returns 'b' (wrong domain, no error)	Exactly one @ required
sender_domain	' a@B.com ' and 'a@b.com\n'	Returns 'b.com ' / 'b.com\n' (whitespace kept in domain)	Strip surrounding whitespace and reject any whitespace inside
first_link	None	AttributeError	Non-string input returns None
first_link	'', 'no link here'	ValueError (substring not found)	Regex search, None when no match
first_link	'nothttp is a word', 'see http'	Returns 'http' (not a link, no error)	Link must start with http:// or https://
first_link	'HTTP://A.COM'	ValueError (search was case sensitive)	Case-insensitive match

Function	Failing input	Original behaviour	Fix
first_link	'hi https://a.com\nnext line'	Returns 'https://a.com\nnext' (only space ended the link)	Link ends at any whitespace (\S+)
shout_ratio	None, 5	TypeError (not iterable)	Non-string input returns 0.0
shout_ratio	'', '123', '!!!'	ZeroDivisionError (no letters)	Return 0.0 when there are no letters

Step 5: Fix the application and run it

Every function now validates its input and returns a safe value for anything invalid: `None` for sender_domain and first_link, and `0.0` for shout_ratio. `main()` prints a clear message instead of crashing. Key changes:

```
import re
_LINK_RE = re.compile(r"https?://\S+", re.IGNORECASE)

def sender_domain(email):
    if not isinstance(email, str):
        return None
    email = email.strip()
    if email.count('@') != 1 or any(c.isspace() for c in email):
        return None
    user, domain = email.split('@')
    if not user or not domain:
        return None
    return domain.lower()

def first_link(text):
    if not isinstance(text, str):
        return None
    match = _LINK_RE.search(text)
    return match.group(0) if match else None

def shout_ratio(text):
    if not isinstance(text, str):
        return 0.0
    letters = [c for c in text if c.isalpha()]
    if not letters:
        return 0.0
    upper = [c for c in letters if c.isupper()]
    return len(upper) / len(letters)
```

Command: `python spam_scoring.py`.

```
PS C:\Users\Epari Subhransi\SE_lab_assignments_PES1UG24CS161\SE_lab_7> python spam_scoring.py
==== Running sender_domain ====
Enter a sender email (e.g. offers@Deals.com): offers@Deals.com
Sender Domain: deals.com

==== Running first_link ====
Enter a message containing a link:  Visit https://google.com today
First Link: https://google.com

==== Running shout_ratio ====
Enter a message (e.g. FREE prize NOW): FREE prize NOW
Uppercase Ratio: 0.5833333333333334
PS C:\Users\Epari Subhransi\SE_lab_assignments_PES1UG24CS161\SE_lab_7>
```

Figure 10: Fixed script with valid input

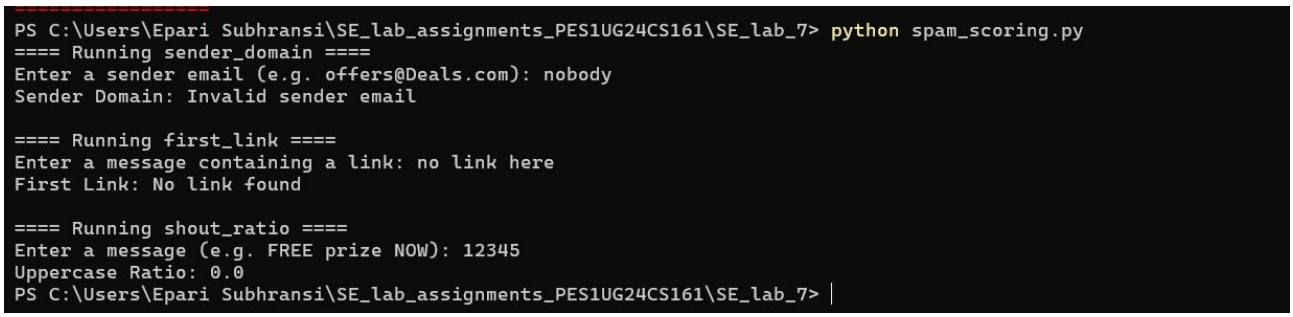


Figure 11: Fixed script handling invalid input (nobody / no link here / 12345) without crashing

Step 6: Re-run test_spam_scoring.py against the fixed script

Command: `python -m pytest test_spam_scoring.py -v`. Result: **14 passed**.

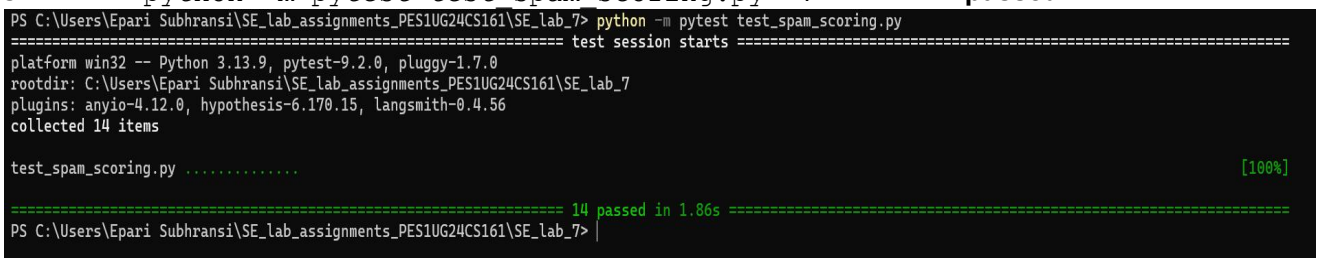


Figure 12: All 14 test cases passing on the fixed script

Results summary

Run	Script	Result
Step 4	Original spam_scoring.py	10 failed, 4 passed
Step 6	Fixed spam_scoring.py	14 passed

Conclusion

Fuzzing with Hypothesis found crashes that normal examples miss: None, empty strings, text with no @, no link or no letters. It also found silent wrong answers: `'a@b@c.com'` giving the wrong domain, `'user@'` giving an empty domain, the word `nothttp` being treated as a link, and a link running past a newline. After adding input validation and returning safe values, all 14 tests pass. The main lesson is that unit tests only check cases the author thought of, while fuzzing checks properties that must hold for every input, so it finds the cases the author never imagined.